



南開大學
Nankai University

数据结构实验报告八

AVL 树

学号：2412266

姓名：杨滢

专业：信息安全

学院：密码与网络安全学院

目录

1 题目一	2
1.1 题目表述	2
1.2 代码 1 的测试用例	2
1.3 思路	2
1.3.1 高度查询	2
1.3.2 AVL 树平衡	3
1.4 代码	3
1.5 测试用例截图	7
1.6 最优算法	8
1.6.1 时间复杂度: $O(1)$	8
1.6.2 空间复杂度: $O(1)$	8
2 题目二	8
2.1 题目表述	8
2.2 代码 2 的测试用例	8
2.3 思路	9
2.3.1 最近叶节点查找	9
2.3.2 插入操作	9
2.3.3 删除操作	9
2.4 代码	9
2.5 测试用例截图	14

1 题目一

1.1 题目表述

编写函数，计算 AVL 树的高度，要求说明该函数是所有算法中最优的。

1.2 代码 1 的测试用例

约定空树高度为 0，非空树高度为根节点到叶子节点的最长路径上的节点数，f 为编写的函数，T 为树。

1. $f(T) \rightarrow 0$ # 空树
2. $\text{insert}(T, 30), f(T) \rightarrow 1$
3. $\text{insert}(T, 40), \text{insert}(T, 50), f(T) \rightarrow 2$
4. $\text{insert}(T, 20), \text{insert}(T, 10), f(T) \rightarrow 3$
5. $\text{insert}(T, 25), f(T) \rightarrow 3$
6. $\text{insert}(T, 60), \text{insert}(T, 70), f(T) \rightarrow 4$
7. $\text{erase}(T, 10), \text{erase}(T, 25), f(T) \rightarrow 3$

1.3 思路

1.3.1 高度查询

插入和删除操作之后都进行高度更新，非空节点高度 = 1 + max(左子树高度, 右子树高度)。

```
1 void updateHeight(Node* node) {  
2     if (node != nullptr) {  
3         node->height = 1 + max(getTreeHeight(node->left),  
4                                 getTreeHeight(node->right));  
5     }  
6 }
```

使用 f(T) 函数直接返回根节点存储的高度值

```
1 int getTreeHeight(Node* root) {  
2     if (root == nullptr) {  
3         return 0;  
4     }  
5     return root->height;  
6 }  
7 int f(Node* T) {  
8     return getTreeHeight(T);  
9 }
```

1.3.2 AVL 树平衡

1. 高度管理:

getHeight() 函数安全地获取节点高度，处理空节点情况。每次插入或删除后更新相关节点的高度。

2. 四种旋转情况

- 左左情况 (LL): 对失衡节点右旋转
- 右右情况 (RR): 对失衡节点左旋转
- 左右情况 (LR): 先左旋左子树，再右旋当前节点
- 右左情况 (RL): 先右旋右子树，再左旋当前节点

1.4 代码

```
1 #include <iostream>
2 #include <fstream>
3 #include <algorithm>
4 #include <string>
5 using namespace std;
6 struct Node {
7     int data;
8     Node* left;
9     Node* right;
10    int height;
11    Node(int val) {
12        data = val;
13        left = right = nullptr;
14        height = 1;
15    }
16 };
17 int getTreeHeight(Node* root) {
18     if (root == nullptr) {
19         return 0;
20     }
21     return root->height;
22 }
23 void updateHeight(Node* node) {
24     if (node != nullptr) {
25         node->height = 1 + max(getTreeHeight(node->left),
26                                getTreeHeight(node->right));
27     }
```

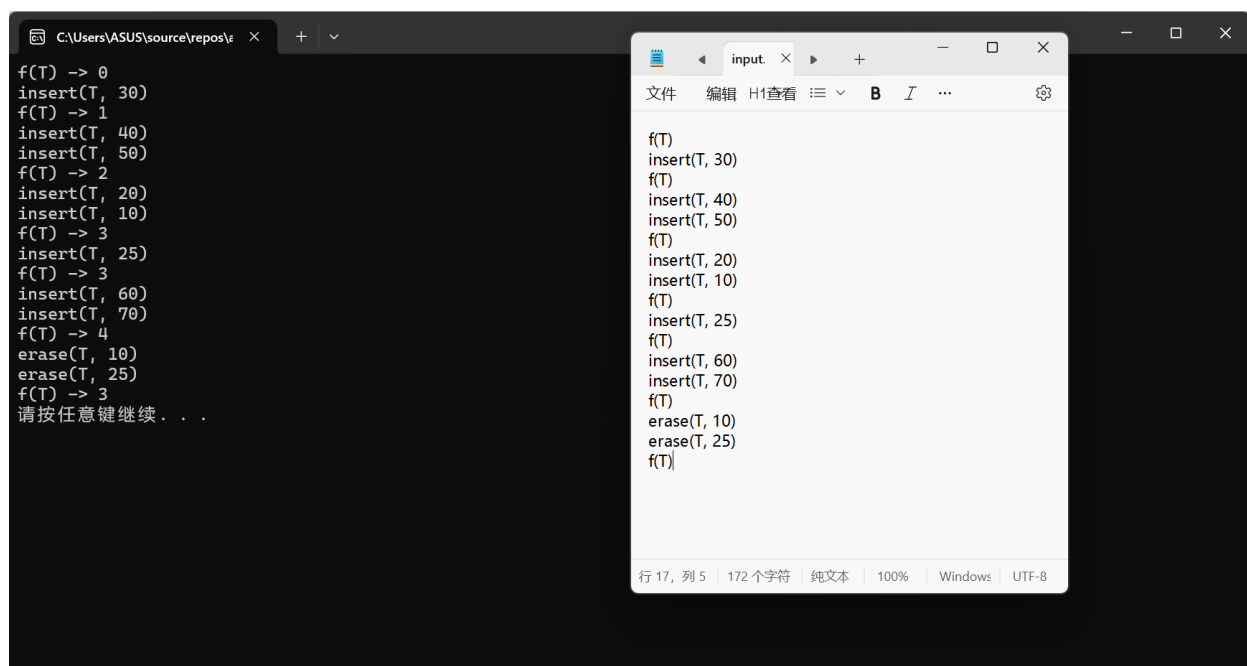
```
28 Node* rotateRight(Node* y) {
29     Node* x = y->left;
30     Node* T2 = x->right;
31     x->right = y;
32     y->left = T2;
33     updateHeight(y);
34     updateHeight(x);
35     return x;
36 }
37 Node* rotateLeft(Node* x) {
38     Node* y = x->right;
39     Node* T2 = y->left;
40     y->left = x;
41     x->right = T2;
42     updateHeight(x);
43     updateHeight(y);
44     return y;
45 }
46 int getBalance(Node* node) {
47     if (node == nullptr)
48         return 0;
49     return getTreeHeight(node->left) - getTreeHeight(node->right);
50 }
51 Node* insertNode(Node* node, int data) {
52     if (node == nullptr) {
53         return new Node(data);
54     }
55     if (data < node->data) {
56         node->left = insertNode(node->left, data);
57     }
58     else if (data > node->data) {
59         node->right = insertNode(node->right, data);
60     }
61     else {
62         return node;
63     }
64     updateHeight(node);
65     int balance = getBalance(node);
66     if (balance > 1 && data < node->left->data) {
67         return rotateRight(node);
68     }
69     if (balance < -1 && data > node->right->data) {
```

```
70     return rotateLeft(node);
71 }
72 if (balance > 1 && data > node->left->data) {
73     node->left = rotateLeft(node->left);
74     return rotateRight(node);
75 }
76 if (balance < -1 && data < node->right->data) {
77     node->right = rotateRight(node->right);
78     return rotateLeft(node);
79 }
80 return node;
81 }
82
83 Node* findMinNode(Node* node) {
84     Node* current = node;
85     while (current->left != nullptr) {
86         current = current->left;
87     }
88     return current;
89 }
90
91 Node* deleteNode(Node* root, int data) {
92     if (root == nullptr) return root;
93
94     if (data < root->data) {
95         root->left = deleteNode(root->left, data);
96     }
97     else if (data > root->data) {
98         root->right = deleteNode(root->right, data);
99     }
100    else {
101        if (root->left == nullptr || root->right == nullptr) {
102            Node* temp = root->left ? root->left : root->right;
103            if (temp == nullptr) {
104                temp = root;
105                root = nullptr;
106            }
107            else {
108                *root = *temp;
109            }
110            delete temp;
111        }
```

```
112     else {
113         Node* temp = findMinNode(root->right);
114         root->data = temp->data;
115         root->right = deleteNode(root->right, temp->data);
116     }
117 }
118 if (root == nullptr) return root;
119 updateHeight(root);
120 int balance = getBalance(root);
121 if (balance > 1 && getBalance(root->left) >= 0) {
122     return rotateRight(root);
123 }
124 else if (balance > 1 && getBalance(root->left) < 0) {
125     root->left = rotateLeft(root->left);
126     return rotateRight(root);
127 }
128 else if (balance < -1 && getBalance(root->right) <= 0) {
129     return rotateLeft(root);
130 }
131 else if (balance < -1 && getBalance(root->right) > 0) {
132     root->right = rotateRight(root->right);
133     return rotateLeft(root);
134 }
135 else
136     return root;
137 }
138 int f(Node* T) {
139     return getTreeHeight(T);
140 }
141 int main() {
142     Node* myTree = nullptr;
143     ifstream inputFile("input.txt");
144     if (!inputFile) {
145         cout << "WRONG" << endl;
146         system("pause");
147         return 1;
148     }
149     string line;
150     while (getline(inputFile, line)) {
151         line.erase(remove(line.begin(), line.end(), ' '), line.end());
152         if (line == "f(T)") {
153             cout << "f(T) -> " << f(myTree) << endl;
```

```
154     }
155     else if (line.find("insert(T,") != string::npos) {
156         size_t start = line.find("insert(T,") + 9;
157         size_t end = line.find(")", start);
158         string valueStr = line.substr(start, end - start);
159         int value = stoi(valueStr);
160         myTree = insertNode(myTree, value);
161         cout << "insert(T, " << value << ")" << endl;
162     }
163     else if (line.find("erase(T,") != string::npos) {
164         size_t start = line.find("erase(T,") + 8;
165         size_t end = line.find(")", start);
166         string valueStr = line.substr(start, end - start);
167         int value = stoi(valueStr);
168         myTree = deleteNode(myTree, value);
169         cout << "erase(T, " << value << ")" << endl;
170     }
171 }
172 inputFile.close();
173 system("pause");
174 return 0;
175 }
```

1.5 测试用例截图



1.6 最优算法

考虑最坏情况：完全不平衡的树，计算 AVL 树高度 n 个节点都需要遍历。

1.6.1 时间复杂度： $O(1)$

$f(T)$ 函数调用 $getTreeHeight(T)$ 函数，直接检查 `nullptr`：

- 空树：1 次比较
- 非空树：1 次比较 + 1 次内存访问（读取 `height` 字段）

时间复杂度恒为 $O(1)$

1.6.2 空间复杂度： $O(1)$

```
1 int f(Node* T) {  
2     return getTreeHeight(T); // 无递归，立即返回  
3 }
```

- 参数传递：1 个指针
- 返回地址：1 个

达到理论下界，是最优算法。

2 题目二

2.1 题目表述

编写函数，返回 AVL 树中距离根节点最近的叶节点的值。

2.2 代码 2 的测试用例

1. $f(T) \rightarrow 0$
2. $\text{insert}(T, 30), f(T) \rightarrow 30$
3. $\text{insert}(T, 40), \text{insert}(T, 50), f(T) \rightarrow 30$
4. $\text{insert}(T, 20), \text{insert}(T, 10), f(T) \rightarrow 50$
5. $\text{insert}(T, 25), f(T) \rightarrow 10$
6. $\text{insert}(T, 60), \text{insert}(T, 70), f(T) \rightarrow 10$
7. $\text{erase}(T, 10), \text{erase}(T, 25), f(T) \rightarrow 20$

2.3 思路

2.3.1 最近叶节点查找

这里使用**广度优先搜索 (BFS)**。

从根节点开始按层次遍历树，返回第一个遇到的叶节点，即最近的叶节点。

2.3.2 插入操作

1. 按 BST 规则递归找到插入位置
2. 创建新节点并链接到树中
3. 回溯更新路径上所有节点的高度
4. 检查并修复平衡性

2.3.3 删除操作

1. 按 BST 规则递归找到要删除的节点
2. 处理三种删除情况：
 - 无子节点：直接删除
 - 一个子节点：用子节点替代
 - 两个子节点：用右子树最小值替代
3. 回溯更新高度并修复平衡

2.4 代码

```
1 #include <iostream>
2 #include <fstream>
3 #include <queue>
4 #include <algorithm>
5 #include <sstream>
6 using namespace std;
7 struct Node {
8     int data;
9     Node* left;
10    Node* right;
11    int height;
12 };
13
14 int getHeight(Node* node) {
15     if (node == nullptr)
16         return 0;
```

```
17     return node->height;
18 }
19
20 Node* createNode(int data) {
21     Node* newNode = new Node();
22     newNode->data = data;
23     newNode->left = nullptr;
24     newNode->right = nullptr;
25     newNode->height = 1;
26     return newNode;
27 }
28
29 Node* rightRotate(Node* y) {
30     Node* x = y->left;
31     Node* T2 = x->right;
32     x->right = y;
33     y->left = T2;
34     y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
35     x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
36     return x;
37 }
38
39 Node* leftRotate(Node* x) {
40     Node* y = x->right;
41     Node* T2 = y->left;
42     y->left = x;
43     x->right = T2;
44     x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
45     y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
46     return y;
47 }
48
49 int getBalance(Node* node) {
50     if (node == nullptr) return 0;
51     return getHeight(node->left) - getHeight(node->right);
52 }
53
54 Node* insertNode(Node* node, int data) {
55     if (node == nullptr)
56         return createNode(data);
57     if (data < node->data)
58         node->left = insertNode(node->left, data);
```

```
59     else if (data > node->data)
60         node->right = insertNode(node->right, data);
61     else
62         return node;
63
64     node->height = 1 + max(getHeight(node->left), getHeight(node->right));
65     int balance = getBalance(node);
66
67     if (balance > 1 && data < node->left->data)
68         return rightRotate(node);
69
70     if (balance < -1 && data > node->right->data)
71         return leftRotate(node);
72
73     if (balance > 1 && data > node->left->data) {
74         node->left = leftRotate(node->left);
75         return rightRotate(node);
76     }
77
78     if (balance < -1 && data < node->right->data) {
79         node->right = rightRotate(node->right);
80         return leftRotate(node);
81     }
82     else
83         return node;
84 }
85
86 Node* findMinNode(Node* node) {
87     Node* current = node;
88     while (current->left != nullptr)
89         current = current->left;
90     return current;
91 }
92
93 Node* deleteNode(Node* root, int data) {
94     if (root == nullptr) return root;
95     if (data < root->data)
96         root->left = deleteNode(root->left, data);
97     else if (data > root->data)
98         root->right = deleteNode(root->right, data);
99     else {
100         if ((root->left == nullptr) || (root->right == nullptr)) {
```

```
101     Node* temp = root->left ? root->left : root->right;
102
103     if (temp == nullptr) {
104         temp = root;
105         root = nullptr;
106     }
107     else {
108         *root = *temp;
109     }
110     delete temp;
111 }
112 else {
113     Node* temp = findMinNode(root->right);
114     root->data = temp->data;
115     root->right = deleteNode(root->right, temp->data);
116 }
117 }
118 if (root == nullptr) return root;
119 root->height = 1 + max(getHeight(root->left), getHeight(root->right));
120 int balance = getBalance(root);
121
122 if (balance > 1 && getBalance(root->left) >= 0)
123     return rightRotate(root);
124
125 else if (balance > 1 && getBalance(root->left) < 0) {
126     root->left = leftRotate(root->left);
127     return rightRotate(root);
128 }
129
130 else if (balance < -1 && getBalance(root->right) <= 0)
131     return leftRotate(root);
132
133 else if (balance < -1 && getBalance(root->right) > 0) {
134     root->right = rightRotate(root->right);
135     return leftRotate(root);
136 }
137 else
138     return root;
139 }
140
141 int findClosestLeaf(Node* root) {
142     if (root == nullptr) return 0;
```

```
143     queue<Node*> q;
144     q.push(root);
145     while (!q.empty()) {
146         Node* current = q.front();
147         q.pop();
148         if (current->left == nullptr && current->right == nullptr) {
149             return current->data;
150         }
151         if (current->left != nullptr) {
152             q.push(current->left);
153         }
154         if (current->right != nullptr) {
155             q.push(current->right);
156         }
157     }
158     return 0;
159 }
160
161 int main() {
162     Node* tree = nullptr;
163     ifstream inputFile("input.txt");
164     string line;
165     while (getline(inputFile, line)) {
166         line.erase(remove(line.begin(), line.end(), ' '), line.end());
167         if (line == "f(T)") {
168             int result = findClosestLeaf(tree);
169             cout << "f(T) -> " << result << endl;
170         }
171         else if (line.find("insert(T,") != string::npos) {
172             size_t start = line.find("insert(T,") + 9;
173             size_t end = line.find(")", start);
174             string dataStr = line.substr(start, end - start);
175             int data = stoi(dataStr);
176             tree = insertNode(tree, data);
177             cout << "insert(T, " << data << ")" << endl;
178         }
179         else if (line.find("erase(T,") != string::npos) {
180             size_t start = line.find("erase(T,") + 8;
181             size_t end = line.find(")", start);
182             string dataStr = line.substr(start, end - start);
183             int data = stoi(dataStr);
184             tree = deleteNode(tree, data);
```

```
185     cout << "erase(T, " << data << ")" << endl;
186 }
187 }
188 inputFile.close();
189 system("pause");
190 return 0;
191 }
```

2.5 测试用例截图

